```
  purchase: ->
    if @info.stock > 0
      post "/json/purchase/#{@purchaseCategory}/#{@name}", (res) =>
        if res.status is "success"
          @info = res.update
          @render()

  mark: ->
    @view.style.border = "1px solid black"

  unmark: ->
    @view.style.border = "none"

# class Camera omitted - see listing 5.3
# class Skateboard omitted - see listing 5.3
class Shop
  constructor: ->
    @view = document.createElement 'input'
    @view.onchange = ->
      Product.find @value
    document.body.appendChild @view
    @render()
    get '/json/list', (data) ->
      for own category of data
        for own name, info of data[category]
          switch category
            when 'camera'
              new Camera name, info
            when 'skateboard'
              new Skateboard name, info

  render: ->
    @view.innerHTML = ""

shop = new Shop
```

The purchase method sends a request and updates the view via render.

The mark and unmark methods make the search result visible on the view.

The Shop class needs to determine which class to construct based on the data.

Notice that some parts of the program are becoming a bit difficult to follow. The view (which manipulates things displayed in the browser) takes up a lot of room in the program and obscures the other parts of the program. There's a way to move those parts of the code somewhere else where they won't get in the way, and you will learn about it later in the chapter. Before moving on, though, there's one more thing you need to know about class properties.

Class property inheritance

If a `Product` class has an existing `find` method, then a `Camera` class that extends `Product` will inherit that existing `find` method:

```
products = [
    name: "Shark repellant"
  ,
```